RosettASM: An Interactive Visualization Environment for High-Level Code Execution and Machine State

Steven Counterman
Student, Department of Math & Computer Science, DeSales University
sc4214@desales.edu
Evan Shimkanon
Instructor, Department of Math & Computer Science, DeSales University
evan.shimkanon@desales.edu

ABSTRACT
In modern computer science education, students often write programs using high-level languages without being exposed to how those same programs execute at the machine level. These languages abstract away machine architecture to facilitate the development of program logic while hiding the interaction with registers, memory, and the program stack. As a result, machine behavior is often treated as a "black box," and students lack accurate mental models of machine state during runtime.

RosettASM is an educational programming environment developed by the authors that bridges the gap between high-level code and machine architecture. This system presents source code, the generated assembly, and register, memory, and stack states concurrently via step-through execution on a graphical interface. RosettASM contains its own custom, simplified, instructional programming language, the associated translation pipeline for target code generation, and a pedagogical interface to reinforce machine architecture concepts. This paper presents the design and prototype implementation of RosettASM.

KEY WORDS
Programming Education, Machine Architecture, Educational Tools

1. Introduction

Computer science education relies heavily on high-level programming languages to introduce students to programming concepts. A challenge with this approach is that these languages abstract away the underlying machine architecture, creating the appearance that they operate independently of the hardware [1]. Consequently, students lack exposure to machine behavior related to their source code and therefore lack appropriate mental models to accompany it.

Constructing accurate knowledge of machine-level execution is integral to understanding structure and control flow within high-level programs [2]. Concepts such as recursion, stack frames, and program flow become more explicit when viewed through the lens of machine-level execution. For example, recursion is often taught as repeatedly calling a function until a base case is reached. However, instruction frequently emphasizes control flow rather than the underlying runtime stack behavior that enables these calls [3].

To address this gap, we developed RosettASM, an interactive platform that allows students to work directly with their own source code. The name "RosettASM" is inspired by the Rosetta Stone, reflecting the system's goal of translating high-level programming constructs to assembly-level machine execution. The suffix "ASM" refers to assembly language. Its accessible didactic environment uncovers the black box by allowing the user to step through their assembly translation and observe runtime state changes in registers, memory, and the program stack. These components reside within the same interface, allowing explicit mapping across abstraction layers and reinforcing sequential reasoning and concrete mental models of execution. The remainder of this paper is organized as follows. Section 1.1 provides background context of work related to this topic. Section 2 describes the system design of RosettASM, including the instructional language, translation pipeline, and visualization interface. Section 3 discusses the advantages, limitations, practical applications, and future work associated with the system.

1.1 Previous Work

Prior research on visual program execution has explored the use of notional machines to help students understand program behavior during execution [4]. Python Tutor, for example, is a widely used web-based visualization system designed to help students step through code and observe program state [5]. While these tools make program execution more visible to beginners, they typically operate at the level of the source language and abstract away the underlying architecture.

Figure 1. Python Tutor visualization of variable state after expression evaluation.

Alternatively, debuggers allow developers to step through program execution, set breakpoints, and inspect program state during runtime. While immensely useful, debuggers such as GNU's GDB [6] are tailored to professional developers with prior knowledge of the underlying architecture and familiarity with development tools. As a result, they tend to have complex interfaces and large feature sets that are not paired with instructional guidance. While they are powerful, debuggers are primarily designed for professional software development rather than programming pedagogy.

RosettASM aims to consolidate program visualization environments and professional debuggers by providing an instructional sandbox environment. By connecting high-level code, generated assembly, and machine state visualization within a single interface, RosettASM provides a learner-friendly setting conducive to reasoning about program execution.

2. System Design

The RosettASM environment includes three main components that support code translation and visualization. The first is a custom instructional programming language that allows students to create their own source code. The second is a translation pipeline responsible for converting the source program into assembly code. The final component is an interactive visualization interface that represents the resulting assembly and machine state. The following subsections describe each component in greater detail.

2.1 Language Design

The primary motivation for a custom programming language is to reduce the cognitive load associated with programming. Many programming languages, such as Java or C-based systems, use similar syntax, making it familiar to students. To avoid unnecessary complexity, it does not utilize a full feature set or libraries. The simplicity allows students to focus on the machine execution represented in the interface rather than the code itself.

The language currently supports the following constructs:

- Variable declaration and assignment
- Arithmetic expressions
- Relational and equality comparisons
- Conditional control flow (if/elif/else)
- Iterative control flow (for, while)
- Flow control (break, continue)
- Commenting

Included is a small statically typed system that supports integer, float, Boolean, and character values to enable basic semantic checking while maintaining simplicity. The syntax is similar to many widely used high-level programming languages. It is semicolon-delimited and block structured, reminiscent of Java. Functionality is intentionally limited, as the language is designed for instructional purposes rather than production development. This allows assignments to focus on program behavior rather than full-scale software projects.

2.2 Translation Pipeline

When a student runs their program in the text editor, the source code is sent to the lexical analyzer. During lexical analysis, the source code is converted into token objects representing keywords, symbols, identifiers, literals, operators, and punctuation. Whitespace and line comments are ignored at this stage, while delimiters such as semicolons, parentheses, and braces are preserved as tokens for parsing.

Using these tokens, the parser evaluates the structure of the program according to the language's syntax rules. It recursively verifies that the code follows these rules and produces Node objects containing relevant metadata. These nodes are nested according to their syntactic relationships to construct an Abstract Syntax Tree (AST), which represents the program's structure and order of evaluation. The AST is then passed to semantic analysis for verification against the language's semantic rules.

During semantic analysis, a symbol table is maintained to track variable declarations and ensure variables are instantiated before use. This stage also verifies type correctness for assignments and expressions. In addition, scope rules are introduced for code blocks, including nested scopes within conditionals and loops. Once these checks are completed, the verified AST is passed to the Three Address Code (TAC) generator.

The TAC generator traverses the AST to produce an intermediate representation resembling assembly instructions. These statements follow the three-address format and use temporary variables to represent intermediate values while remaining independent of specific hardware registers. In contrast to typical compilers, optimization is intentionally avoided so that intermediate steps remain explicit, allowing students to

observe how higher-level expressions translate into lower-level operations.

The TAC is then passed to the target code generator. At this stage, variable lifetimes and next-use information are analyzed to determine register allocation and stack usage. Values that cannot be assigned to registers are temporarily stored on the program stack. The resulting assembly instructions are then displayed in the appropriate panel of the user interface.

2.3 Visualization Interface

The RosettASM environment presents the results of the translation pipeline through an interactive user interface designed to expose machine behavior alongside the source program.

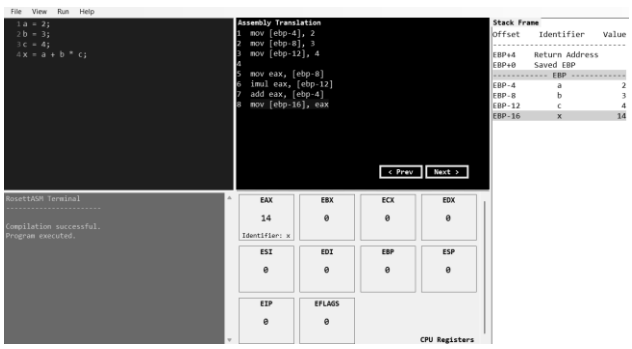

Figure 2. RosettASM interface displaying example program from Figure 1.

Students write code directly in an integrated editor and can select individual lines to observe how they are translated into assembly instructions. Selecting a line of assembly reveals the associated machine state, allowing students to observe how variables move between registers and memory as execution progresses. The interface displays multiple panels simultaneously to provide a complete synchronous view of program execution. The editor panel allows students to write and select source code, while a terminal panel reports compilation and runtime errors. A central panel displays the generated assembly instructions, and additional panels visualize register contents and the program stack.

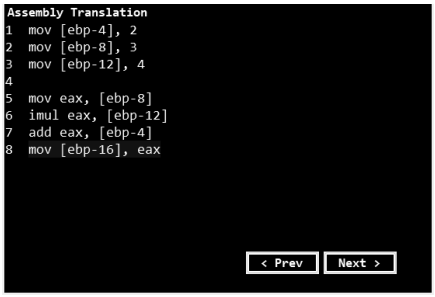

Figure 3. Assembly instruction translation panel with sample data.

The stack visualization shows variables relative to the base pointer, allowing students to observe how values are stored and updated during execution.

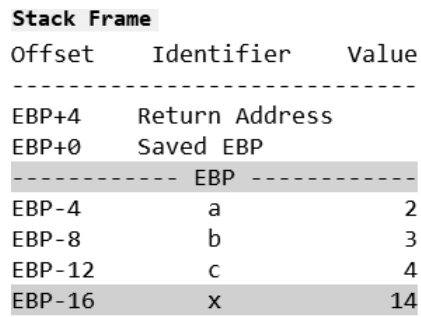

Figure 4. Stack visualization panel with sample data.

As students step through the program, the interface dynamically updates to reflect changes in machine state. Variables that move into registers are highlighted, while values written back to the stack are visually indicated.

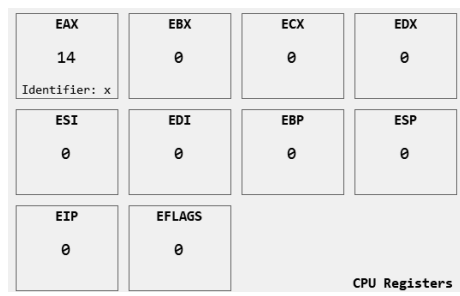

Figure 5. CPU register panel with sample data.

Register values are displayed in decimal form for readability, and stack entries show the variable name, stored value, and offset relative to the base pointer. These visual cues help students track how program execution modifies machine state through each step. This simultaneous presentation of source code, assembly, and machine state helps students build an intuitive understanding of how high-level programs execute at the hardware level.

3. Conclusion

RosettASM was developed as an educational environment meant to expose the relationship between high-level code and the machine architecture that executes it. By combining a simplified instructional language, a translation pipeline, and an interactive visualization interface, this system allows students to observe the transformation of their source code into machine state changes during runtime. This approach supports constructivist learning by making the normally hidden processes of program execution visible to learners. The following subsections discuss the advantages, limitations, potential educational applications, and future work associated with RosettASM.

3.1 Advantages

The primary advantage RosettASM offers is the bridging of abstraction layers. Concepts like stack frames, registers, data movement, and control flow are taught using architecture visualizations. The separation of computer science courses into those focused on high-level programming and those focused on computer architecture can create the false impression that these concepts are independent. This system consolidates these principles and presents computing constructs as a unified pipeline. Typical development environments do not present source code and execution state simultaneously for evaluating machine behavior. While these environments hide the interaction, RosettASM focuses on constructing the mental models associated with removing abstraction.

Another advantage is that seeing machine state reinforces constructivist learning concepts. This allows foundational machine components to be learned first and high-level constructs to be built with concrete understanding of these components. Stepping through execution allows students to directly see their own source code in action instead of interacting with assembly in isolation. This helps capture explicit machine state the moment each instruction runs. As a result, learners can observe what operations occur for each line of source code.

RosettASM provides an avenue to lower the barrier to machine-level concepts. Allowing students to write high-level syntax instead of assembly reduces the prerequisite knowledge required to begin exploring program execution. Assembly languages express computation in terms of registers and instructions rather than variables and human-adjacent linguistic systems. Without requiring immediate familiarity with assembly language, students can observe how common programming constructs translate into machine-level behavior. This simplification promotes independent exploration, as students can utilize concepts they already understand from prior programming experience.

Explicit visualization of execution is paramount to constructing accurate knowledge. To accomplish this, non-optimized translation from intermediate representation to assembly keeps intermediate steps explicit. Typically, these steps are merged for compilers for efficiency, but RosettASM preserves them so students can observe atomic operations. In showing register and stack information simultaneously, data movement and stack storage is explicit for student observation. This helps learners understand how variables are stored in memory, their usage, and stack changes during execution.

3.2 Limitations

The custom programming language has explicit limitations in functionality. It does not currently support advanced language constructs typically provided by high-level counterparts used in development. For example, user-defined functions are not supported, so recursion cannot yet be exemplified. Object-oriented design and data structures are also not supported at this time. At minimum, user-defined functions would be implemented before broader classroom deployment. The current version represents the initial prototype and serves as a framework for future expansion.

Secondly, the system supports a single architecture. The current implementation focuses solely on x86 32-bit assembly. This restriction contrasts the common usage of x86-64 in modern systems. Additional modules for x86-64, ARM, and RISC-V have been considered for further capability extension.

The visualizations provided represent a simplified machine model. Certain machine behaviors such as cache interactions and heap memory are not currently represented. Additionally, nuanced aspects of execution are simplified, such as character variables utilizing four-byte allocations to maintain consistent stack alignment. The system is designed as an instructional tool rather than a production environment.

Ultimately, RosettASM is intended to ease students into low-level concepts, rather than replacing deeper study of these topics. Advanced students may eventually outgrow the environment as they move toward full systems programming. There exists no replacement for learning real assembly programming when deeper comprehension of architecture is required. The system is intended to be an introductory bridge to machine-level concepts rather than a replacement.

3.3 Practical Applications

The RosettASM environment may support instruction in several areas of computer science education. In computer architecture or organization courses, students are often introduced to registers, memory, and other low-level components for the first time. Novice programmers frequently struggle to understand the hidden runtime behavior of programs and the machine state that underlies program execution [7]. RosettASM provides a way to expose students to assembly instructions and machine state without requiring them to write assembly code directly. This reduces the friction associated with learning low-level concepts, while still allowing students to observe how programs interact with underlying hardware. Initial classroom deployment of RosettASM is planned at a local university, with a potential trial run at a local high school, to evaluate its effectiveness in introducing machine architecture components to both university and secondary-level students.

This system may also benefit introductory programming courses once students are familiar with fundamental concepts such as variables, conditionals, and loops. Since

the custom programming language uses familiar high-level syntax, students can write programs without extensive prior experience while still observing the assembly translation process and machine-level execution. This lower barrier to entry encourages independent exploration of machine-level behavior without requiring students to first learn the complexities of assembly programming. Early exposure to these relationships can help students construct mental models of program execution before encountering more advanced programming language concepts. RosettASM also allows students to explore concepts that are often difficult to internalize, including stack frames, data movement, and control flow. By presenting the source code, generated assembly, and machine state together, students can observe how program execution affects registers and memory at each step.

3.4 Future Work

RosettASM's compiler was developed to promote modular expansion. The next planned addition is support for user-defined functions. The addition of functions would enable the visualization of function calls and recursion. This would allow students to observe parameters pushed to the stack and the resulting changes to program state. Recursion functionality would allow students to observe how stack frames accumulate and unwind during program execution.

The next logical expansion for architecture would be development of an x86-64 module to align with modern systems. Additional architectures such as ARM or RISC-V may also be considered to broaden the system's applicability. Utilization of different target assembly code reinforces understanding of foundational assembly constructs.

Future visualization development includes representing heap memory in addition to the program stack. Additional alternative visualizations could include explicit function stack frames and recursive execution behavior. These additions would allow students to observe a wider range of runtime behaviors beyond simple stack-based variable storage.

Lastly, RosettASM may be expanded for use in additional computer science courses. For example, extensions could support instruction in operating systems by visualizing concepts such as scheduling or synchronization. The system could also expose portions of the compiler pipeline itself to help students understand how source code is transformed into lower-level representations. Continued development of RosettASM aims to further strengthen the connection between student and machine.

References:

[1] R. Bryant and D. O'Hallaron, Computer systems: a programmer's perspective 2nd edition (Boston, MA: Pearson, 2011).

[2] M. Ben-Ari, Constructivism in computer science education, *Proc. 29th SIGCSE Technical Symposium on Computer Science Education*, Atlanta, GA, 1998, 257-261.

[3] C. Rinderknecht, A survey on the teaching and learning of recursive programming, *Informatics in Education*, *13*(1), 2014, 87-119.

[4] J. Sorva, *Visual program simulation in introductory programming education* (Aalto University, FI, 2012).

[5] P.J. Guo, Online Python tutor: embeddable web-based program visualization for cs education, *Proc. ACM Technical Symposium on Computer Science Education*, New York, NY, 2013, 579-584.

[6] Free Software Foundation, GDB: the GNU project debugger, www.gnu.org/software/gdb, December 12th 2025.

[7] J. Sorva, V. Karavirta, & L. Malmi, A review of generic program visualization systems for introductory programming education, *ACM Transactions on Computing Education*, *13*(4), 2013, 1-64.